S2S-06: Step 6 — Integrate: Cloud, Dashboards, and Workflows

> **Series:** S2S — SaaS to SaaS Migration | **Notebook:** 6 of 9 | **Phase:** Upgrade | **Step:** Integrate | **Created:** March 2026 | **Last Updated:** 10/27/2026

Overview

With agents reporting to the target tenant and core configuration imported (Step 5), this step reconnects everything that depends on external systems: cloud integrations, dashboards, workflows, notification channels, synthetic monitors, and extensions. These integrations were deliberately deferred until agents were providing entity context — without entities, cloud metrics have no topology to attach to, dashboards show no data, and workflows have no triggers.

This step completes the Upgrade phase. After this step, the target tenant is fully operational and the migration enters the Run phase (Steps 7–9).

S2S Migration Framework

Phase	Steps	Focus
Plan	1. Discover → 2. Strategize → 3. Design	Understand what exists, choose approach, design target
Upgrade	4. Prepare → 5. Execute → **6. Integrate**	Build target, move config, connect integrations
Run	7. Expand → 8. Enable → 9. Optimize	Roll out agents, enable teams, tune and decommission

> **You are here: Step 6 — Integrate.** Agents are reporting to the target tenant and core configuration is deployed (Step 5). Now you reconnect cloud integrations, migrate dashboards and workflows, and restore all external connections.

Table of Contents

1. Cloud Integration Migration
2. Cloud Transformation Scenarios
3. Dashboard Migration
4. Workflow Migration
5. Notification Integration Migration
6. Synthetic Monitor Migration
7. Extension Migration
8. Step Completion Checklist

Prerequisites

Requirement	Details
Step 5 Complete	All agents reporting to target tenant, configuration imported, validation queries passing

Requirement	Details
Cloud Provider Access	AWS: rights to deploy CloudFormation stacks in each monitored account. Azure: rights to create a service principal and assign it on each monitored subscription (Microsoft Entra ID). GCP: IAM admin
Target Tenant Access	API token with `WriteConfig`, `settings.write`, `credentialVault.write` scopes
Notification Channel Access	Admin access to Slack, PagerDuty, ServiceNow, Teams, or email systems
Synthetic Private Locations	Network access from private location hosts to target tenant
Extension Host	ActiveGate group for remote extensions; SQL extensions may alternatively run in Kubernetes via Dynatrace Operator — verify per extension

Order of Operations

This notebook covers **operations 7–8** of the Upgrade phase:

#	Operation	Section	Dependency
1	Provision target tenant and configure SSO/IAM	Step 4	Step 3 design deliverables
2	Export configuration from source tenant	Step 4	Source tenant access
3	Deploy ActiveGates and prepare K8s operators	Step 4	Target tenant provisioned
4	Import configuration to target tenant	Step 5	Operations 1–3 complete
5	Redirect agents to target tenant	Step 5	Configuration imported
6	Validate data flow in target tenant	Step 5	Agents redirected
7	**Reconnect cloud integrations**	Sections 1–2	Agents reporting
8	**Migrate dashboards, workflows, and extensions**	Sections 3–7	Integrations connected

> **Bold** operations are covered in this notebook. Operations 1–6 were completed in Steps 4 and 5.

1. Cloud Integration Migration

Cloud connections are tenant-specific. Credentials, trust relationships and monitoring scopes are recreated in the target — Monaco can deploy cloud integration **configurations** but **cannot export credentials**.

Two generations of cloud monitoring exist, and this section is written **Latest first**:

- **Latest Dynatrace — Cloud Platform Monitoring** (AWS and Azure connections, viewed in the Clouds app). Polling runs inside Dynatrace SaaS: *"no need to deploy ActiveGate compute resources for metric polling"*.
- **Dynatrace Classic integrations** — the fallback, marked as such below, for a target that cannot use the Latest connection yet.

Azure — Azure Cloud Platform Monitoring (Latest)

Component	Target action
Azure connection	Create a new connection **from the target**, with a **new, dedicated** service principal — never the source environment's principal
Metrics and topology	Polled by Dynatrace SaaS; no ActiveGate is deployed for polling
Logs	SaaS-based ingest via Azure Event Hubs — activity logs, resource logs, Entra ID audit logs, Defender for Cloud alerts. Microsoft 365 workload audit is collected separately — see the note below

Component	Target action
Events	Event Grid system topics forwarding resource lifecycle events

> **A Microsoft 365 audit collector moves on its own.** If the source collects Exchange, SharePoint or Teams audit through the Office 365 Management Activity API (FAQ-27), that collector is not part of the Azure connection and is not recreated with it. Point its ingest URL and token at the target, and keep its checkpoint: FAQ-27's collectors restart at the last 24 hours without one. Move the bucket, pipeline and masking rules it relies on with the rest of the log configuration.
>
> **Do not monitor a subscription twice.** The Azure connection docs: *"Do not onboard Azure subscriptions already monitored by the classic Azure integration, and avoid monitoring the same subscription across multiple Azure connections—both increase the risk of API throttling and service interruptions."* In a tenant move, that rules out a long overlap in which both the source and the target poll the same subscription. Switch **per subscription**: create the target's connection for a subscription, confirm its resources appear in the target, then remove that subscription from the source's integration (classic or Latest) in the same change window.

AWS — AWS Cloud Platform Monitoring (Latest)

Component	Target action
AWS connection	Create a new connection **from the target**; it is deployed through CloudFormation into each monitored account
Metrics and topology	Polled by Dynatrace SaaS; no ActiveGate is deployed for polling
Logs	Subscribe CloudWatch log groups to the Firehose streams the target's connection generates

The AWS onboarding page read for this update carries no equivalent of the Azure double-monitoring warning. Treat a period in which both environments poll the same AWS account as something to keep short and to confirm with your Dynatrace account team (cost and API quota), not as a documented limit either way.

> **Retiring AWS for Azure?** The target needs its **own** AWS connection for the transition — AWS workloads keep running until their wave moves, and they must be visible in the target before the source environment can go. That AWS connection is itself decommissioned last. **S2S-94** is the ordered runbook.
>
> **Sources:**
>
> - Azure Cloud Platform Monitoring (DT docs) — *"The new Azure Cloud Platform Monitoring is fully managed by Dynatrace SaaS—no need to deploy ActiveGate compute resources for metric polling within your Azure environment."*
> - Create your first Azure connection (DT docs) — *"Use a dedicated service principal exclusively for Dynatrace. Do not share it across Dynatrace environments or use it for other non-Dynatrace workloads."*
> - AWS Cloud Platform Monitoring (DT docs) — *"All AWS connection creation methods are powered by CloudFormation as Infrastructure-as-Code (IaC) engine."*; *"Subscribe CloudWatch log groups to auto-generated Firehose streams for immediate ingestion and analysis."*

Classic Fallback — Dynatrace Classic Integrations

Use these only where the target cannot use the Latest connections yet.

AWS (classic):

Component	Source	Target Action
IAM Role	`arn:aws:iam::role/DynatraceMonitoring`	Create new role with target tenant external ID in trust policy
CloudWatch metrics	Configured per region	Recreate with same region scope
Log Forwarder	Lambda function forwarding CloudWatch Logs	Deploy new Lambda stack with target tenant ingest URL

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::&lt;dynatrace-aws-account-id-from-connection-wizard&gt;:root"
      },
      "Action": "sts:AssumeRole",
      "Condition": {
        "StringEquals": {
          "sts:ExternalId": "&lt;target-tenant-external-id&gt;"
        }
      }
    }
  ]
}
```

> **Copy the principal and external ID from the target tenant's AWS connection setup** — an IAM principal ARN needs the 12-digit account ID (`arn:aws:iam::<account-id>:root`), and AWS rejects a policy without it. See **CLOUD-02** for the connection model. The external ID changes per tenant; remove the source tenant's external ID after validation.

Azure (classic):

Component	Source	Target Action
Service principal	App registration used by the classic integration	Create a new one for the target
Azure Monitor	Subscription-scoped metrics	Configure the same subscriptions in the target — then remove them from the source (no double monitoring)
Log forwarder	Classic Azure log forwarder	Redeploy pointing at the target. It ingests directly through the Cluster API by default; an ActiveGate is needed only if you choose not to use direct ingest

> **Classic log-forwarder cut-over timing.** *"Logs older than 24 hours are rejected (considered too old by the Dynatrace log ingest endpoint)."* A forwarder repointed more than a day after its logs were produced cannot backfill them — switch it inside the change window, not after an outage.
>
> **Sources:** Set up the Azure log forwarder (DT docs, Dynatrace Classic) — *"Azure log forwarding is performed directly through Cluster API. If you don't want to use direct ingest through the Cluster API, you have to use an existing ActiveGate for log ingestion."*

GCP Integration

Not re-verified in this update — see the CLOUD series for the current connection model.

Component	Source	Target Action
Service Account	JSON key for Cloud Monitoring	Generate new key for same service account, or create new SA
Pub/Sub	Log forwarding via Pub/Sub subscription	Create new subscription for target tenant
Cloud Monitoring	Project-scoped metrics	Configure same projects in target tenant

Validate Cloud Integration Data

After configuring cloud integrations in the target tenant, verify data is flowing:

```
// Target tenant: check for detected problems indicating integration issues
//
// Corrected 08/12/2026: the cell sorted by `timestamp` AFTER a `fields` projection that dropped it,
// so it failed with FIELD_DOES_NOT_EXIST. dt.davis.problems does carry `timestamp` — `fields` had
// simply removed it. Keep the sort key in the projection.
fetch dt.davis.problems, from:-2h
| filter contains(event.name, "cloud") or contains(event.name, "integration") or contains(event.name, "AWS") or
contains(event.name, "Azure") or contains(event.name, "GCP")
| fields timestamp, display_id, event.name, event.status, event.category
| sort timestamp desc
| limit 10
```

```
// Target tenant: verify deployment events are being captured
fetch events, from:-24h
| filter event.kind == "DAVIS_EVENT" and event.type == "CUSTOM_DEPLOYMENT"
| summarize deployment_count = count()
| fieldsAdd status = if(deployment_count &gt; 0, then: "Deployment events flowing", else: "No deployment events — check
CI/CD integration")
```

2. Cloud Transformation Scenarios

Two different changes hide under "cloud transformation" (**S2S-01** §1): the Dynatrace environment moving to a cluster on another cloud (a tenant move — this whole series), and the **monitored workloads** moving to another cloud. When the workloads move, not only the credentials change — the monitoring stack and the data it produces change too.

AWS → Azure Workloads (Latest Dynatrace)

Component	AWS (leaving)	Azure (arriving)
Connection	AWS connection (CloudFormation)	Azure connection (dedicated service principal)
Cloud metrics	`cloud.aws.*` — e.g. `cloud.aws.ec2.CPUUtilization.By.InstanceId`	`cloud.azure.*` — e.g. `cloud.azure.microsoft_compute.virtualmachines.PercentageCPU`
Logs	Firehose	Event Hubs

Component	AWS (leaving)	Azure (arriving)
Container platform	EKS	AKS
Hosts (OneAgent)	`cloud.provider == "aws"`	`cloud.provider == "azure"`
Serverless	Lambda	Azure Functions

The metric keys above were read with the `metrics` command on the validation tenant (09/28/2026).

> **Dashboard impact:** plan to **recreate** cloud-specific dashboard tiles, not migrate them. Dashboards built on OneAgent data (`dt.host.*`, services, spans, logs) carry across clouds; dashboards built on `cloud.aws.*` metrics do not have an Azure equivalent key to remap to.

The Reverse Direction

Azure → AWS is the mirror image: create the AWS connection through CloudFormation, move log ingest from Event Hubs to Firehose, and rewrite `cloud.azure.*` tiles against `cloud.aws.*`.

Validate Dual-Cloud Coverage

During the window, the target should see both clouds — through the connections (cloud resources) and through OneAgent (hosts):

```
// Target tenant: cloud resources per provider (connections) — run during the dual-cloud window
smartscapeNodes "*", from:-2h
| filter startsWith(type, "AWS_") or startsWith(type, "AZURE_")
| fieldsAdd provider = if(startsWith(type, "AWS_"), then: "aws", else: "azure")
| summarize {resources = count(), resource_types = countDistinct(type)}, by:{provider}

// A provider with zero resources means its connection is missing or not yet polling.
// Pair it with the host query in S2S-94 (hosts by cloud.provider and region) to see OneAgent
// coverage next to connection coverage.
```

3. Dashboard Migration

Dashboards were imported in Step 5 as part of the Monaco deploy. This section covers the post-import remediation required to make dashboards functional.

Dashboard Types and Migration Path

Type	Monaco Type	Migration Path	Post-Import Work
Classic dashboards	`api` (dashboard)	Monaco deploy	Entity ID remapping, ownership reset
Gen3 dashboards	`document`	Monaco deploy	Search the exported `document` JSON for `HOST-` / `SERVICE-` / `PROCESS_GROUP-` literals (S2S-05 §4 grep) — DQL tiles that filter on entity IDs render empty after migration
Gen3 notebooks	`document`	Monaco deploy	Same entity-ID search as dashboards — a DQL cell filtering on an entity ID returns nothing, without an error

Entity ID Remediation

Classic dashboards frequently contain hardcoded entity IDs in tile filters. These must be updated to reference entities in the target tenant:

Pattern	Source	Target
Tile entity filter	`entityId("HOST-1A2B3C4D")`	`tag("env:production")`
SLO reference	`sloId("slo-abc-123")`	New SLO ID from target tenant
Management zone filter	`mzId(12345)`	New MZ ID from target tenant

> **Gen3 advantage — when the DQL is written for it:** DQL that filters on names or tags survives migration; DQL that filters on entity IDs (`dt.entity.host == "HOST-…"`) does not, and fails silently. If you rebuild classic dashboards as Gen3 documents, filter on names or tags rather than remapping entity IDs.

Ownership Reset

Dashboards imported via Monaco are owned by the API token user. Reassign ownership to the appropriate teams:

Dashboard Category	New Owner	Method
Platform overview	Platform team	Manual reassignment in UI
Application dashboards	App team leads	Manual reassignment in UI
Executive dashboards	Dashboard admin group	Manual reassignment in UI
Shared dashboards	Set to "shared" visibility	Bulk update via API

4. Workflow Migration

Workflows (Dynatrace Workflows, formerly AutomationEngine) require special handling because they contain actor permissions, triggers, and external integrations that are tenant-specific.

Workflow Components

Component	Portable?	Migration Notes
Workflow definition	Yes	Export via Monaco (`automation`) or Terraform
Trigger configuration	Partial	Dynatrace Intelligence triggers reference entity IDs — update selectors
Actor permissions	No	Reassign actor (the identity that executes the workflow) in target tenant
External connections	No	Webhook URLs, API keys, OAuth tokens must be recreated

Component	Portable?	Migration Notes
JavaScript/Python actions	Yes	Code is portable; external URLs may need updating

Terraform Export/Import

```
# Export workflows from the source tenant (source-tenant credentials in the environment).
# The workflow resource is excluded from a default export, so name it explicitly.
terraform-provider-dynatrace -export dynatrace_automation_workflow

# Edit the generated configuration for the target tenant:
# - Change trigger entity selectors
# - Update webhook URLs and connection references
# - Reassign actor

# Apply with target-tenant credentials
terraform init &amp;amp;&amp;amp; terraform apply
```

`terraform import` is not an export: it binds an existing object into Terraform state and writes no configuration, so importing from the source and applying to the target copies nothing. The provider's `-export` mode writes the configuration.

> **Sources:** dynatrace_automation_workflow (Dynatrace GitHub) — *"This resource is excluded by default in the export utility, please explicitly specify the resource to retrieve existing configuration."*

Trigger Reconfiguration

Trigger Type	Migration Action
detected problem	Update entity selectors (replace hardcoded IDs with tags)
Event	Update event type filters if namespace/service names changed
Schedule (cron)	Port as-is — cron expressions are tenant-independent
Manual	Port as-is

Actor Permissions

Every workflow has an **actor** — the identity (user or service account) under which the workflow executes. The actor is authorized by IAM policies in the target tenant, not by token scopes:

```
Source actor: user@company.com (admin in source tenant)
Target actor: Same user, or dedicated service account
Required access: IAM permissions for each action the workflow runs (see the IAM series)
```

> **Best practice:** Use a dedicated **service account** as the workflow actor instead of a personal user account. Service accounts survive employee turnover and can be scoped precisely.

5. Notification Integration Migration

Notification integrations connect Dynatrace alerting to external systems. Each integration type has different migration requirements.

Integration Type Matrix

Integration	Configuration Migrated via Monaco?	Credential Update Required?	Notes
Email	Yes	No	Recipient addresses are portable

Integration	Configuration Migrated via Monaco?	Credential Update Required?	Notes
Slack	Yes (structure)	Yes	Webhook URL must be updated to new channel/workspace
Microsoft Teams	Yes (structure)	Yes	Incoming webhook URL must be recreated
PagerDuty	Yes (structure)	Yes	Integration key must be updated
ServiceNow	Yes (structure)	Yes	Instance URL, credentials must be updated
Webhook (generic)	Yes (structure)	Yes	URL and auth headers must be verified
Jira	Yes (structure)	Yes	Project keys and credentials must be updated
OpsGenie	Yes (structure)	Yes	API key must be updated

Update Checklist

For each notification integration:

Step	Action	Validation
1	Verify integration was imported by Monaco	Integration visible in target tenant UI
2	Update credentials/API keys	Store in credential vault
3	Update webhook URLs if environment-specific	Test URL is reachable
4	Send test notification	Verify delivery in target system
5	Validate alerting profile linkage	Profile references correct notification

> **Test every notification channel.** Send a test alert from the target tenant to each notification integration. Do not assume that a successful import means the notification will work — credentials, URLs, and permissions may have changed.

6. Synthetic Monitor Migration

Synthetic monitors are imported via Monaco but require post-import validation for location assignments and credential references.

Monitor Types

Type	Monaco Migration	Post-Import Work
HTTP monitors	Full export/import	Update credential vault references
Browser monitors	Full export/import	Verify script actions still work
Browser click-path	Full export/import	Re-record if target application URL changed

Location Assignment

Location Type	Migration Notes
Public locations	Global — same location IDs across tenants. Port as-is.
Private locations	Tenant-specific. Deploy new private location AG in target tenant.

Private Location Migration

Step	Action
1	Create private synthetic location in target tenant UI
2	Deploy ActiveGate with synthetic capability at that location
3	Update monitor location assignments to use new location ID
4	Verify monitors execute from the new location

Validate Synthetic Monitors

After migration, verify synthetic monitors are executing in the target tenant:

```
// Target tenant: count active synthetic monitors, by monitor type
smartscapeNodes "BROWSER_MONITOR"
| summarize monitor_count = count()
| fieldsAdd monitor_type = "browser"
| append [smartscapeNodes "HTTP_MONITOR" | summarize monitor_count = count() | fieldsAdd monitor_type = "http"]
| append [smartscapeNodes "NETWORK_AVAILABILITY_MONITOR" | summarize monitor_count = count() | fieldsAdd monitor_type =
"network availability"]
| fieldsAdd validation = "Compare each type against the source tenant count"

// Smartscape (preferred, verified 07/2026): dt.entity.synthetic_test maps to the BROWSER_MONITOR
// node (individual steps are a separate BROWSER_MONITOR_STEP node). HTTP monitors are HTTP_MONITOR,
// multi-protocol monitors NETWORK_AVAILABILITY_MONITOR, private locations SYNTHETIC_LOCATION. This
// corrects an earlier note here that claimed no Smartscape equivalent existed. Unlike ActiveGate,
// `fetch dt.entity.synthetic_test` does still work and remains a genuine fallback — it reads the
// classic entity store, which can retain entities Smartscape (live topology) no longer lists.
// Classic fallback: fetch dt.entity.synthetic_test | summarize monitor_count = count()
// Count all three monitor types: a browser-only count passes a parity check even if no HTTP
// or network availability monitor migrated.
```

7. Extension Migration

Extensions 2.0 provide monitoring for technologies not covered by OneAgent (databases, network devices, cloud APIs). Remote extensions run on an ActiveGate group; SQL monitoring extensions can alternatively run in Kubernetes through Dynatrace Operator.

Extension Migration Steps

Step	Action	Notes
1	Inventory extensions from source tenant	List all active extensions and their versions
2	Verify extension availability in target tenant	Extensions are published to the Dynatrace Hub — verify same version exists
3	Install extensions in target tenant	Via Hub, API, or Terraform (`dynatrace_hub_extension_active_version`)
4	Configure monitoring settings	Recreate endpoint configurations
5	Assign to the ActiveGate group (or Kubernetes runtime) the extension supports	Verify per extension
6	Validate data collection	Check for new metrics from extension

Extension Types

Type	Migration Path
Hub extensions	Install from Hub in target tenant — configuration is portable via Monaco

Type	Migration Path
Custom extensions	Upload `.zip` to target tenant, recreate configuration
JMX extensions	Port JMX configuration file, update endpoint references
SNMP extensions	Port SNMP configuration, update device IP addresses if changed

> **Plan the extension runtime per extension.** Remote extensions run on an ActiveGate group; SQL monitoring extensions can alternatively run in Kubernetes through Dynatrace Operator (see the K8S series). Make sure the runtime each extension needs exists in the target tenant (as prepared in Step 4, Section 4). Monaco does not export extension installations; Terraform can install and activate them (`dynatrace_hub_extension_active_version`) and configure them (`dynatrace_hub_extension_v2_config`).
>
> **Sources:** Extensions (DT docs) — *"Run SQL monitoring extensions on Kubernetes using Dynatrace Operator."*, dynatrace_hub_extension_active_version (Dynatrace GitHub) — *"In case the extension has not yet gotten installed for the specified version the installation happens automatically."*

8. Step Completion Checklist

Before proceeding to Step 7 (Expand), verify all integration tasks are complete:

Deliverable	Status	Owner	Notes
AWS connection created from the target	☐	Cloud / Platform	CloudFormation stack deployed per account (classic fallback: IAM role trust policy updated)
Azure connection created from the target	☐	Cloud / Platform	Dedicated service principal; each subscription removed from the source in the same window (no double monitoring)
GCP integration configured	☐	Cloud / Platform	Service account key generated, Cloud Monitoring connected
Log ingest active	☐	Cloud / Platform	Firehose (AWS) / Event Hubs (Azure) / Pub/Sub (GCP) sending to the target; classic forwarders switched inside the 24-hour window
Classic dashboards remediated	☐	Platform	Entity IDs remapped, ownership reassigned
Gen3 dashboards validated	☐	Platform	DQL queries returning data
Workflows migrated	☐	Platform	Actors assigned, triggers reconfigured
Notification integrations tested	☐	Platform	Test alert sent to each channel
Synthetic monitors executing	☐	Platform	All monitors running from correct locations
Extensions installed and collecting	☐	Platform	Extension metrics visible in target tenant
No critical detected problems	☐	Platform	All migration-related problems resolved or documented

> **Phase transition:** Completing this checklist ends the **Upgrade** phase. The target tenant is now fully operational. The **Run** phase (Steps 7–9) focuses on expanding agent coverage, enabling teams, and optimizing the target tenant.

Next Step

Continue to **S2S-07: Step 7 — Expand: Agent Rollout and Coverage** to expand agent coverage to remaining hosts, enable team access, and begin the Run phase.

Completed	Next	Remaining
~~1. Discover~~ → ~~2. Strategize~~ → ~~3. Design~~ → ~~4. Prepare~~ → ~~5. Execute~~ → ~~6. Integrate~~	**7. Expand**	8. Enable → 9. Optimize

---

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.